```
  after_create do |order|
    logger.info "Order #{order.id} created"
  end
  protected
  def normalize_credit_card_number
    self.cc_number.gsub!(/[-\s]/, '')
  end
end
```

You can specify multiple handlers for the same callback. They will generally be invoked in the order they're specified unless a handler throws `:abort`, in which case the callback chain is broken early.

Alternatively, you can define the callback instance methods using callback objects, inline methods (using a proc), or inline eval methods (using a string). See the online documentation for more details.[154]

Grouping Related Callbacks Together

If you have a group of related callbacks, it may be convenient to group them into a separate handler class. These handlers can be shared between multiple models. A handler class is simply a class that defines callback methods (`before_save`, `after_create`, and so on). Create the source files for these handler classes in `app/models`.

In the model object that uses the handler, you create an instance of this handler class and pass that instance to the various callback declarations. A couple of examples will make this clearer.

If our application uses credit cards in multiple places, we might want to share our `normalize_credit_card_number` method across multiple models. To do that, we'd extract the method into its own class and name it after the event we want it to handle. This method will receive a single parameter, the model object that generated the callback:

```
class CreditCardCallbacks
```